Microservicios y APIs Escalables

Universidad de Palermo · 2026

Sprint 12

API Gateway con Spring Cloud Gateway

Clínica Veterinaria "Patitas Felices"

Contexto y objetivo del Sprint

Al final del Sprint 11, la arquitectura tiene cuatro microservicios funcionales, comunicados entre sí vía OpenFeign y registrados en Eureka. Sin embargo, el cliente externo (una app móvil, un frontend, Postman) necesita conocer **cuatro puertos distintos** para acceder a cada servicio: :8081, :8082, :8083, :8084.

Este esquema escala mal: si un servicio cambia de puerto, todos los clientes deben actualizarse. Además, no existe un punto centralizado para aplicar autenticación, logging o rate limiting.

Objetivo: agregar un **API Gateway** en el puerto :8080 que actúe como único punto de entrada. El gateway recibe las peticiones, las enruta dinámicamente a través de Eureka, y los clientes externos solo necesitan conocer :8080.

Arquitectura antes y después del Gateway

Antes (Sprint 11) — cliente conoce cada puerto

```
Cliente
  ├── GET  :8081/api/duenos     → ms-dueno
  ├── GET  :8082/api/mascotas   → ms-mascota
  ├── GET  :8083/api/veterinarios → ms-veterinario
  └── POST :8084/api/turnos     → ms-turno
```

Después (Sprint 12) — cliente solo habla con :8080

```
Cliente
  └── :8080 (api-gateway)
        ├── /api/duenos/**      → lb://ms-dueno
        ├── /api/mascotas/**    → lb://ms-mascota
        ├── /api/veterinarios/**→ lb://ms-veterinario
        └── /api/turnos/**      → lb://ms-turno
```

> **lb://** es el prefijo de Spring Cloud Load Balancer. El gateway resuelve el nombre contra Eureka y balancea entre instancias disponibles en cada llamada.

Responsabilidades del API Gateway

Responsabilidad	Descripción	¿En este sprint?
Enrutamiento dinámico	Redirige según path a microservicio destino vía Eureka	Sí

Responsabilidad	Descripción	¿En este sprint?
Load balancing	Distribuye carga entre instancias del mismo servicio	Sí (automático)
Logging centralizado	Registra cada request/response en un solo lugar	Sí (filter)
CORS	Permite llamadas desde frontends en otros dominios	Sí (config)
Autenticación JWT	Valida token antes de pasar al microservicio	Sprint 14
Rate limiting	Limita peticiones por IP o usuario	Sprint 14
Circuit breaker	Fallback si un microservicio no responde	Sprint 13

Pasos del Sprint

PASO 1 — Agregar módulo api-gateway al proyecto multi-módulo

En el `pom.xml` raíz, agregar el módulo dentro de `<modules>`:

```
    <modules>
        <module>ms-dueno</module>
        <module>ms-mascota</module>
        <module>ms-veterinario</module>
        <module>ms-turno</module>
        <module>eureka-server</module>
        <module>config-server</module>
        <module>api-gateway</module>   ← agregar
    </modules>
```

Crear la carpeta `api-gateway/` en la raíz del proyecto.

PASO 2 — Crear el pom.xml del módulo api-gateway

Crear `api-gateway/pom.xml`:

```
    <?xml version="1.0" encoding="UTF-8"?>
    <project xmlns="http://maven.apache.org/POM/4.0.0"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
             http://maven.apache.org/xsd/maven-4.0.0.xsd">
        <modelVersion>4.0.0</modelVersion>
        <parent>
            <groupId>com.patitas</groupId>
            <artifactId>patitas-felices</artifactId>
            <version>1.0.0</version>
        </parent>
        <artifactId>api-gateway</artifactId>
        <dependencies>
            <dependency>
                <groupId>org.springframework.cloud</groupId>
                <artifactId>spring-cloud-starter-gateway</artifactId>
            </dependency>
            <dependency>
```

```
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-actuator</artifactId>
        </dependency>
    </dependencies>
</project>
```

⚠️ Spring Cloud Gateway usa WebFlux (reactivo), NO el Servlet stack tradicional. NO agregar spring-boot-starter-web — son incompatibles. Si aparece el error "Spring MVC found on classpath", es porque se mezcló web con gateway.

PASO 3 — Crear la clase principal ApiGatewayApplication

Crear la estructura de paquetes `src/main/java/com/patitas/gateway/` y dentro la clase principal:

```
package com.patitas.gateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ApiGatewayApplication {
    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }
}
```

No requiere ninguna anotación especial — el gateway se activa automáticamente por la dependencia en el pom.

PASO 4 — Configurar las rutas en application.yml

Crear `api-gateway/src/main/resources/application.yml`:

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true            # auto-descubre servicios de Eureka
          lower-case-service-id: true
      routes:
        - id: ms-dueno
          uri: lb://ms-dueno
          predicates:
            - Path=/api/duenos/**
```

```
        - id: ms-mascota
          uri: lb://ms-mascota
          predicates:
            - Path=/api/mascotas/**

        - id: ms-veterinario
          uri: lb://ms-veterinario
          predicates:
            - Path=/api/veterinarios/**

        - id: ms-turno
          uri: lb://ms-turno
          predicates:
            - Path=/api/turnos/**

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
  instance:
    prefer-ip-address: true

management:
  endpoints:
    web:
      exposure:
        include: health,info,gateway
  endpoint:
    gateway:
      enabled: true

# NO agregar spring.jpa.database-platform
```

Cada entrada en `routes` tiene tres partes: **id** (nombre único), **uri** (destino — el nombre en Eureka con prefijo lb://), **predicates** (condición para activar esta ruta — en este caso el path).

PASO 5 — Agregar configuración CORS en el Gateway

Para que un frontend pueda llamar al gateway desde otro dominio, agregar la configuración CORS en `application.yml`, dentro de `spring.cloud.gateway`:

```
spring:
  cloud:
    gateway:
      globalcors:
        cors-configurations:
          '[/**]':
            allowedOrigins: "*"
            allowedMethods:
              - GET
              - POST
              - PUT
```

```
              - DELETE
              - OPTIONS
            allowedHeaders: "*"
            allowCredentials: false
```

⚠️ En producción, reemplazar allowedOrigins: "*" por el dominio real del frontend (ej. https://patitas.com). El wildcard "*" solo es aceptable en desarrollo.

PASO 6 — Crear un GlobalFilter para logging centralizado

Crear la clase `LoggingFilter.java` en el paquete `com.patitas.gateway.filters`. Los filtros del Gateway son reactivos (Mono/Flux):

```
package com.patitas.gateway.filters;

import org.springframework.cloud.gateway.filter.GlobalFilter;
import org.springframework.cloud.gateway.filter.GatewayFilterChain;
import org.springframework.core.Ordered;
import org.springframework.stereotype.Component;
import org.springframework.web.server.ServerWebExchange;
import reactor.core.publisher.Mono;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import java.time.Instant;

@Component
public class LoggingFilter implements GlobalFilter, Ordered {

    private static final Logger log =
        LoggerFactory.getLogger(LoggingFilter.class);

    @Override
    public Mono<Void> filter(ServerWebExchange exchange,
                             GatewayFilterChain chain) {
        long start = Instant.now().toEpochMilli();
        String method = exchange.getRequest().getMethod().name();
        String path   = exchange.getRequest().getPath().toString();

        log.info("[GATEWAY] --> {} {}", method, path);

        return chain.filter(exchange).then(Mono.fromRunnable(() -> {
            long duration = Instant.now().toEpochMilli() - start;
            int status = exchange.getResponse()
                                 .getStatusCode().value();
            log.info("[GATEWAY] <-- {} {} {} {}ms",
                     method, path, status, duration);
        }));
    }

    @Override
    public int getOrder() { return -1; } // ejecutar primero
}
```

getOrder() = -1: asegura que este filtro corre antes que los demás. Cuanto menor el número, mayor la prioridad.

PASO 7 — Primer arranque y verificación de rutas

Orden de arranque (siempre el mismo):

```
# Terminal 1
cd config-server && mvn spring-boot:run

# Terminal 2 (esperar que config-server esté UP)
cd eureka-server && mvn spring-boot:run

# Terminales 3-6 (en cualquier orden, esperar eureka)
cd ms-dueno      && mvn spring-boot:run
cd ms-mascota    && mvn spring-boot:run
cd ms-veterinario && mvn spring-boot:run
cd ms-turno      && mvn spring-boot:run

# Terminal 7
cd api-gateway   && mvn spring-boot:run
```

Verificar en Eureka http://localhost:8761:

Deben aparecer: MS-DUENO, MS-MASCOTA, MS-VETERINARIO, MS-TURNO, API-GATEWAY (5 instancias).

Verificar rutas del gateway:

```
GET http://localhost:8080/actuator/gateway/routes
→ JSON con las 4 rutas configuradas
```

Probar enrutamiento end-to-end:

```
# Antes usabas :8081 — ahora solo :8080
GET  http://localhost:8080/api/duenos
POST http://localhost:8080/api/duenos
     { "nombre":"Carlos", "apellido":"Ruiz",
       "email":"carlos@mail.com", "telefono":"1122334455" }

GET  http://localhost:8080/api/mascotas
POST http://localhost:8080/api/turnos
     { "fechaHora":"2026-11-15T10:00:00",
       "motivo":"Vacunación", "mascotaId":1,
       "veterinarioId":1 }
```

✅ Si los microservicios responden correctamente a través de :8080, el gateway está funcionando.

PASO 8 — Agregar ruta con filtro de reescritura de path (opcional avanzado)

Si en el futuro se quiere exponer el gateway con un prefijo versionado (`/v1/duenos` en vez de `/api/duenos`), se usa el filtro `RewritePath`:

```
- id: ms-dueno-v1
  uri: lb://ms-dueno
  predicates:
    - Path=/v1/duenos/**
```

```
    filters:
      - RewritePath=/v1/duenos/(?<segment>.*),
                    /api/duenos/${segment}
```

Así el cliente llama a `/v1/duenos/1` y el gateway reescribe a `/api/duenos/1` antes de enrutar. Útil para versionar la API pública sin cambiar los microservicios internos.

PASO 9 — Agregar el api-gateway a patitas-config (Config Server)

Para que el gateway reciba su configuración del Config Server en producción, crear `api-gateway.yml` en el repositorio `patitas-config`:

```
# patitas-config/api-gateway.yml
server:
  port: 8080

spring:
  cloud:
    gateway:
      discovery:
        locator:
          enabled: true
          lower-case-service-id: true
      globalcors:
        cors-configurations:
          '[/**]':
            allowedOrigins: "*"
            allowedMethods: [GET, POST, PUT, DELETE, OPTIONS]
            allowedHeaders: "*"

management:
  endpoints:
    web:
      exposure:
        include: health,info,gateway
```

Actualizar `api-gateway/src/main/resources/application.yml` para usar bootstrap config (igual que ms-dueno en Sprint 10):

```
spring:
  application:
    name: api-gateway
  config:
    import: "configserver:http://localhost:8888"
```

Resumen de rutas del API Gateway

Path externo (:8080)	Microservicio destino	Puerto interno
/api/duenos/**	ms-dueno	8081
/api/mascotas/**	ms-mascota	8082
/api/veterinarios/**	ms-veterinario	8083

Path externo (:8080)	Microservicio destino	Puerto interno
/api/turnos/**	ms-turno	8084
/actuator/**	api-gateway (propio)	8080

Arquitectura completa al cierre del Sprint 12



> **Regla de oro:** a partir de este sprint, **ningún cliente externo** llama directamente a :8081-8084. Todo pasa por :8080. Los puertos internos son un detalle de implementación.

Vista previa — Sprint 13: Resiliencia con Resilience4j

¿Qué pasa si ms-mascota se cae mientras ms-turno intenta validar? Sin circuit breaker, ms-turno espera el timeout completo (5 s) y devuelve 503. Con **Resilience4j** se configura un circuit breaker: después de N fallos seguidos el circuito "abre" y se retorna un *fallback* inmediato. También se cubren retry (reintentos automáticos) y rate limiter.